Be sure to explore the chat interface's boundaries and what you can ask for from the TMDB service. For example, try asking for a list of genres for movies or television shows. This service is a good first try, but we can perhaps do better, as we'll see in the next section.

5.6 Thinking semantically when writing semantic services

Now we've seen an excellent demonstration of converting an API into a semantic service interface. As it is, the functions return the titles of the top movies and television shows currently playing. However, by just returning the titles, we're limiting the ability of the LLM to parse the results on its own.

Therefore, we'll create a v2 version of `TMDbService` to correct this and return the results as JSON strings. Open the file `tmdb_v2.py` in VS Code, and scroll down to the `get_top_movies_by_genre` function.

Listing 5.25 `tmdb_v2.py` (`get_top_movies_by_genre`)

```
def get_top_movies_by_genre(self, genre: str) -> str:
        print_function_call()
        genre_id = self.get_movie_genre_id(genre)
        if genre_id:
            #same code …

            return json.dumps(filtered_movies)     #1
        else:
            return ""
```

#1 Now returns a filtered list as a JSON string

Now open `SK_service_chat.py` in VS Code, and comment and uncomment the line shown in listing 5.26. This will then use version 2 of the `TMDbService` that outputs results as full JSON documents in a single string.

Listing 5.26 `SK_service_chat.py` (modifying imports)

```
#from skills.Movies.tmdb import TMDbService     #1
from skills.Movies.tmdb_v2 import TMDbService     #2
```

#1 Comment out this line.
#2 Uncomment this line to use version 2 of the service.